Automated Rice Leaf Disease Classification Using Deep Learning: A Comparative Study of a Custom CNN and EfficientNetB0

Machine Learning Phase III Project Report

Submitted By

Amulya Jagaluru Manjunath

Matriculation Number: 64537267

Master of Science (M.Sc.) Data Science

Supervisor

Prof. Raja Hashim Ali

University of Europe for Applied Sciences (UE)
Potsdam Campus

Contents

Graphics Abstract

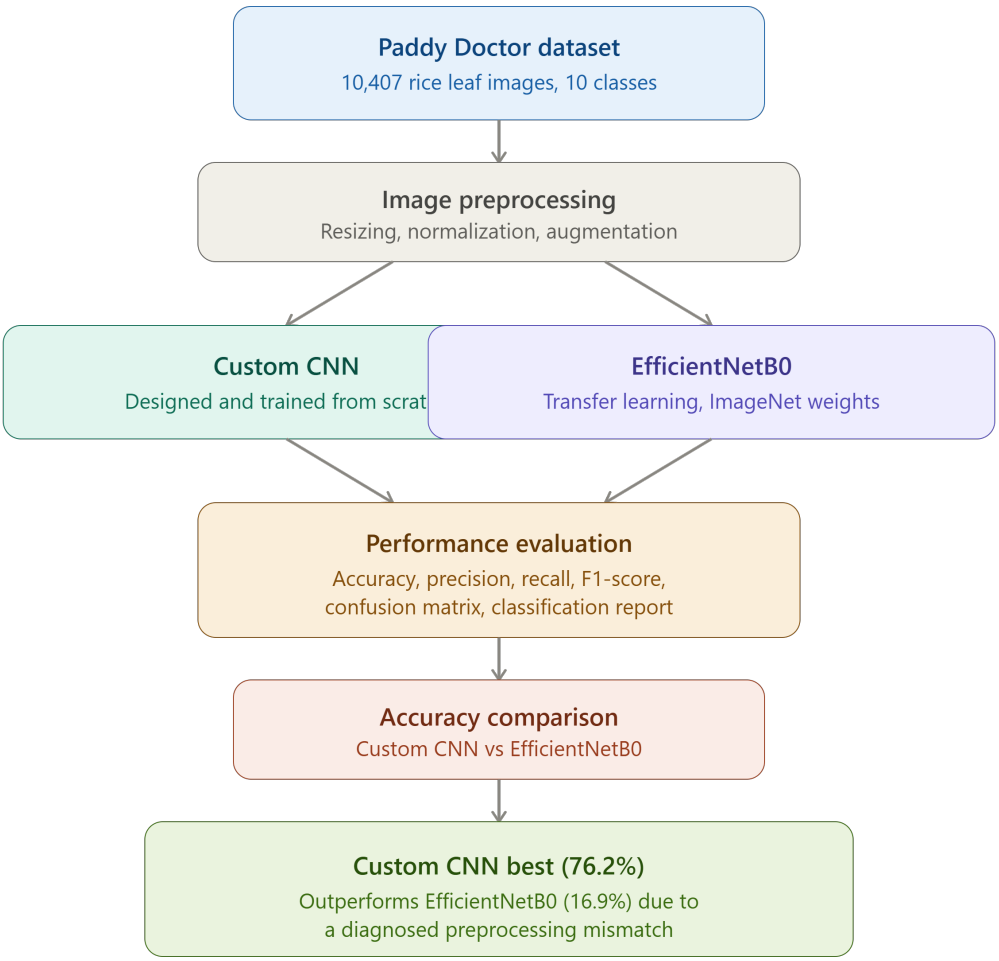

Figure 1: Graphical abstract summarizing the proposed rice leaf disease classification framework

Highlights

- Developed an automated rice leaf disease classification framework using deep learning.
- Implemented and evaluated a Custom CNN architecture for ten-class rice leaf disease classification.
- Applied transfer learning using the EfficientNetB0 architecture pretrained on ImageNet.
- Evaluated both models using accuracy, precision, recall, F1-score, confusion matrices, and classification reports.
- Custom CNN achieved a test accuracy of 76.2%, substantially outperforming EfficientNetB0 (16.9%).
- Diagnosed and explained the root cause of the transfer learning model's underperformance as an input-preprocessing mismatch rather than a fundamental architectural limitation.
- Identified concrete future steps, including corrected transfer learning experiments and explainability analysis, needed to fairly benchmark pretrained architectures on this task.

Abstract

Rice leaf diseases really are a major cause of yield loss in paddy cultivation, and yes manual diagnosis by agronomists just doesn't scale to the total volume of smallholder farms in rice growing regions. Lately, advances in Artificial Intelligence, Computer Vision, and deep learning have made it practical to build automated leaf disease classification systems directly from leaf photographs, not just lab work. In this research, deep learning is looked at for classifying rice leaf images into ten categories, nine are disease types and one is a healthy, or normal, class.

Two experiments were carried out using the publicly accessible Paddy Doctor dataset, with 10,407 labeled rice leaf images. In the comparisons, two deep learning methods were used, first a Custom Convolutional Neural Network (CNN) trained from scratch, and then an EfficientNetB0 transfer learning model, pretrained on ImageNet, where the convolutional backbone is frozen. The experiment routine also involves image pre-processing, normalization, data augmentation, training, and then evaluation of outcomes which are pretty much checked across multiple steps. For assessment, both models were judged with accuracy, precision, recall, F1-score, plus classification reports and confusion matrices.

In the experimental results section, it seems the Custom CNN got a notably higher test accuracy (76.2%) compared to the EfficientNetB0 transfer learning model (16.9%). When the classification report was analyzed , it looked like the EfficientNetB0 was basically stuck predicting only the majority class, and later a more careful inspection traced the issue to a pixel scaling mismatch, between the data pipeline and what the pretrained backbone was actually expecting in terms of input range . So it wasnt about some hard, basic inability of transfer learning to handle this task, more like the inputs were a bit off.

Therefore , it might be said that with the experimental setup used here , the Custom CNN turned out to be the more effective and the more consistently trained model. Still, a corrected transfer learning experiment has to be run first before any broad statement can be made about which approach is more valuable in general.

Overall, the research, kinda indicates that deep learning image classification systems can function as a solid base for automated rice disease monitoring, for real. The study is one of the contributions toward building smarter agricultural monitoring systems , since it evaluates both custom and transfer learning methods, and it also offers a candid, diagnosed account of an experiment that didn't work out, instead of just leaving it out. Moving forward , further work could try corrected transfer learning pipelines , better class imbalance handling, and more transparent explainable AI methods, specifically for rice disease detection.

Keywords: Rice Leaf Disease Detection, Deep Learning, Convolutional Neural Networks, Transfer Learning, EfficientNetB0, Computer Vision.

1 Introduction

1.1 Application Background and Practical Importance

Rice (*Oryza sativa*) is one of the most important staple food crops worldwide, feeding more than half of the global population, and also acting as an economic backbone for agriculture across large parts of Asia. But rice cultivation is, at the same time, persistently threatened by leaf diseases like blast bacterial blight , brown spot, and tungro—these can really cut down the yield and in some cases endanger food security, especially if they're not detected, and managed early enough. Like with many other plant illnesses climate variability, monoculture farming, and the way pathogens move through dense paddy fields have turned disease outbreaks into a constant worry for farmers and agricultural extension services.

The usual approach for spotting rice leaf disease depends on manual checking by trained agronomists or folks that have lots of field experience. It can be quite accurate but still has a few drawbacks , it's quite labor- intensive and it does not really grow with large farms or remote areas where experts are hard to reach. Also the diagnostic precision can shift a lot based on who's looking, the lighting situation. And honestly some symptoms look alike, so the visual cues can get confused. When diagnosis comes late, treatment also starts late, so the disease keeps moving and spreading , before anyone can do a corrective step like a focused pesticide application.

The recent advances in computer vision and artificial intelligence kinda opened up new avenues for using automated, image based plant disease detection. For example, smartphone cameras, drone mounted cameras, and fixed field cameras are now able to capture huge amounts of leaf imagery, and this data can get processed automatically with machine learning methods, instead of depending just on manual inspection. When you build automated systems from this kind of visual information, they can potentially make disease screening faster, more consistent, and easier for people to access, which helps with better informed agricultural decisions. Machine learning technologies, and in particular CNNs, have shown really strong performance when it comes to image recognition. So developing an effective deep learning rice disease detection system is a really crucial step, for improving food security, and also for supporting more resilient, tech supported agriculture.

1.2 Machine Learning and Computer Vision Context

The introduction of machine learning tech has kind of... revolutionized the way computers interpret visual data, by letting them learn patterns from examples rather than strictly sticking to hand coded rules. In computer vision, machine learning is used for image plus video analysis, basically to run several operations like object recognition,

classification, segmentation and even anomaly detection.

Normally, in older image analysis workflows, people relied on pre-defined features made by domain experts. Those features might be color histograms, texture descriptors, edge detection tools, and also shape based attributes, among other things . Even with all the success traditional methods showed in many real world scenarios, they can still be a bit fragile and not very adaptive when you move to new environmental conditions, changes in illumination, partial occlusions, and also when the database gets really large.

Because of that , there is a noticeable shift toward deep learning, mainly for extracting meaningful features on its own automatically, instead of manually designing them every time.

The Convolutional Neural Network (CNN) can be viewed like one of the main inventions in modern computer vision, and it ends up serving as a foundation for image classification tasks. Essentially, a CNN is an artificial neural network that is built to learn layered or nested patterns in visual images through multiple stages of convolutional filters. These early, lower-level layers tend to pick up straightforward cues such as edges, corners, and textures, while the upper, deeper layers go after the more elaborate patterns, shapes, and arrangements. So in a way it's like CNN gradually learns hierarchical features from visual inputs, meaning it can sort or label images based on visual patterns, without someone having to do manual feature extraction. Compared with traditional machine learning methods, CNN usually delivers strong predictive accuracy, plus it stays rather flexible across a variety of image datasets.

In the current study, the rice leaf disease detection issue is basically an image classification matter, like yes, the whole idea is to classify images into one of ten categories nine disease types, plus the healthy normal class. The visual hints tied to each class may vary a lot depending on the lighting situation, the quality of the images, the camera viewpoints used, the background scenes, and also the stage where the disease is progressing. So, using CNN algorithms seems quite suitable here because they can learn separate visual features that then help in distinguishing different disease lesion patterns from healthy leaf tissue. Therefore, in line with the feature learning strength of CNNs, it is planned to develop a practical image classification system designed for automated rice disease monitoring.

1.3 Transfer Learning for the Selected Problem

Transfer learning has turned out to be a pretty efficient path for building a solid deep learning model, especially when computation is constrained or when there is not enough domain data available. For example, when you train a deep neural network from the ground up, you typically need millions of labeled images, large computing resources, and a lot of time. In many practical situations, like rice disease detection, gathering that

many carefully annotated images is not only expensive, but also kind of painfully slow. Because of that, deep learning models that rely solely on task-focused data can struggle with overfitting and end up showing weak generalization. Transfer learning fixes a lot of this by reusing knowledge learned from massive benchmark datasets, then applying it to a new but related problem.

Convolutional neural networks get trained on enormous datasets like ImageNet, which basically includes millions of images spread across thousands of object classes. During that whole training phase, the network ends up learning general visual cues such as edges, surface texture, shapes, and even whole objects. What it learns in those layers, can then be moved to other tasks by swapping out the classification head of the model so it matches the target dataset. This kind of transfer learning tends to happen in two steps, first feature extraction, where the pre-trained layers stay put and do not get modified at all. Then comes fine-tuning, where you unfreeze some of the network layers and retrain them again using the new domain specific data. Transfer learning has gotten really common in computer vision because the resulting model usually predicts better than a network that is initialized randomly and trained from scratch, it's kind of the practical advantage people want.

For this analysis, EfficientNetB0 is picked, to be used in transfer learning, and to be set beside the Custom CNN architecture. EfficientNetB0 is part of the EfficientNet family, it relies on a compound scaling approach to balance depth, width, and the input size, which helps it reach solid accuracy using relatively few parameters when you compare it to older designs. This model has performed really well on many image classification datasets, and it has also been used successfully for agricultural monitoring, medical imaging, object detection, and remote sensing in real workflows. So with EfficientNetB0 included in the experiment plan, the goal of this study is to see whether transfer learning can improve rice leaf disease classification accuracy, when compared to a custom built CNN model.

1.4 Explainable AI and Trustworthiness

The rising use of deep learning models in decision making tasks has kind of resulted in a higher need for interpretability and transparency. Even if deep neural networks are very accurate when it comes to making predictions, they are often still described as "black boxes" because the processes behind them are hardly understandable, for people in general. In an agricultural context like rice disease detection , accuracy alone is not really enough, since there are real economic implications when predictions go wrong. For example, missing an active disease, or sending out a false alarm, would end up causing delayed treatment wasted resources, or even unnecessary pesticide application. So, being able to see and grasp how the prediction is made becomes a useful, almost essential, consideration for the creation of reliable artificial intelligence systems.

Explainable Artificial Intelligence (XAI) methods were introduced, in a kind of way to give more visibility into what deep neural networks are really doing, specifically how their decisions get formed. Tools like Grad-CAM provide heatmap visualizations that highlight the image regions that seem to drive the particular prediction the most. In a similar manner, Shapely Additive explanations (SHAP) give insight into what actually contributes to the final classification outcome, in terms of image areas and also relevant features. So it becomes possible to check whether the model pays attention to meaningful visual patterns, or instead gets distracted by background details, which is not ideal. This kind of thing is very useful for bias discovery , shortcut learning detection , and other related issues too.

Even though the current study mostly focuses on building and evaluating CNN-based image classification algorithms for rice leaf disease detection, the explainability part is still, kind of crucial for practical use. It would be good to be confident that the model truly concentrates on the diseased parts of the leaf rather than on unrelated regions. Unfortunately, the explainability techniques , such as Grad-CAM and SHAP visualizations, were not used in the present work. There were certain limitations , and also because the main goal of this paper is the comparison of models. Still, adding explainability methods later could become a worthwhile direction for future research.

1.5 Research Gap

Due to recent breakthroughs in deep learning there have been quite a few improvements in the image classification area, that can be used for different purposes, like environmental monitoring and spotting crop illnesses. A bunch of researchers have relied on convolutional neural networks, and also transfer learning models to detect rice and plant leaf diseases from images captured using surveillance cameras, drones , and even mobile phones. Even if many of these papers report good classification results, there are still certain problems sitting in the existing literature. For instance, quite a few authors seem to just pick one specific model and stick with it without really comparing its performance against other deep learning architectures. Then it becomes hard, or almost impossible, to tell what the results are actually driven by, whether it is the model picked, or some other factor such as the dataset used or the experimental arrangement. Also, a large share of studies focus only on overall accuracy, and they do not really dig into per class measures like precision, recall, and F1 score.

Another issue that shows up from earlier researches is that the performance of transfer learning, when used for classification of rice disease images, is not analyzed in a properly detailed way. While pre-trained models such as ResNet, MobileNet and EfficientNet have, in many cases, proven to be powerful tools for common computer vision tasks, their edge over hand-crafted CNNs for the specific problem of rice disease detection is

sometimes not really clear. On one side, some authors report strong gains using transfer learning; yet, in other published benchmarks, the pretrained features seem to transfer less effectively to fine-grained, domain specific agricultural classification, than they do to more generic object recognition tasks. Also, the practical reasons behind a transfer learning model that ends up underperforming are seldom diagnosed in detail by most papers, so their conclusions become a bit less useful for practitioners who might meet similar, implementation related issues.

This study tries to cover those gaps by putting together a comparative rice leaf disease classification framework, using two different approaches – one is a Custom CNN model and the other is an EfficientNetB0 transfer learning model. The experiment contrasts the performance of these two methods while keeping everything aligned: the same data set, the same data preprocessing pipeline, the same training method, and the same metrics. Beyond accuracy, the study also examines other performance metrics such as precision, recall, F1-score, confusion matrix values, and classification reports. In addition, instead of only treating an underperforming transfer learning outcome as a plain negative result, this work performs a more detailed root-cause diagnosis of why it happened. That way, the methodological transparency is improved, which is something that is rarely documented in the published literature about this subject.

1.6 Problem Statement

Accurately classifying rice leaf images into multiple disease categories is still really hard, mainly because a few diseases look kinda similar just from the leaves , plus there is the usual mess of lighting and backgrounds when the photos come from actual fields. On top of that, the publicly available datasets have a class imbalance problem too, where some diseases appear with way fewer images than others, like less often seen or just under collected. When classification is wrong, or even arrives late, the farmer can get inappropriate, or delayed treatment, and that really affects crop yield directly.

Here, the issue we're dealing with is the classification of rice leaf images into ten categories: nine disease classes and one healthy normal class. The study is trying to see if an image classification model, built with deep learning techniques, can actually learn visual features that make those categories separable in a useful way. Specifically, the work compares a Custom CNN model with an EfficientNetB0 transfer learning model. Both are evaluated on the publicly available Paddy Doctor dataset. This study also wants to look at the predictive power of each model, and to find which one ends up being more effective , both in prediction ability and practical application.

1.7 Aim and Objectives

Aim

The goal of the suggested study is to make an automatic rice leaf disease classifier, using deep learning methods , sort of like a system that just works on its own. the intention is to check whether Convolutional Neural Network algorithms, or CNN for short, can be used properly for automatically telling apart ten different rice leaf disease categories. in other words, we want to see if these convolutional models can recognize the visual signs reliably . This way, the research can try to contribute toward smarter rice disease classifiers by using computer vision algorithms.

Objectives

1. To explore the feasibility of applying deep learning algorithms to automatically classify rice leaf disease images.
2. To preprocess and prepare the publicly available Paddy Doctor dataset to be used for building and testing models.
3. To build and implement a custom Convolutional Neural Network (CNN) model to classify rice leaf images into ten disease categories.
4. To implement the transfer learning technique by building an EfficientNetB0 model tailored to rice leaf disease classification.
5. To measure the effectiveness of the models on various measures including accuracy, precision, recall, F1-Score, and confusion matrices.
6. To determine which is the most effective approach in predicting rice leaf disease classifications from images, and to diagnose any unexpected results.
7. To discuss the practical implications, limitations, and future prospects of deep learning-based rice disease detection systems.

1.8 Research Questions

The study is guided by the following research questions:

1. Can a Custom Convolutional Neural Network (CNN) accurately classify rice leaf images into the correct disease category?
2. How does the performance of a transfer learning model (EfficientNetB0) compare with a Custom CNN for rice leaf disease classification?

3. Which model provides better classification accuracy, precision, recall, and F1-score for rice leaf disease detection?

4. Which disease classes are most difficult to classify correctly, and why?

5. What are the strengths and limitations of the developed deep learning-based rice leaf disease classification framework?

1.9 Contributions and Novelty

The current research will add more, to the existing literature, on deep learning based agricultural monitoring through the development and evaluation of an automated rice leaf disease classification system that is kinda focused on real world use. This work is especially aimed at sorting rice leaf images into ten distinct categories by using both a Custom CNN and EfficientNetB0 deep learning frameworks with a transfer learning approach. Instead of doing just one setup, the experiment explores two different frameworks within the same research track , so it can reveal something about how well each one detects rice leaf diseases. Also, the use of open source data plus a reproducible methodology, gives extra credibility and practical value, for the overall study.

The biggest contribution of this study is the comparative analysis between a Custom CNN and an EfficientNetB0 transfer learning model, all under the same conditions. These conditions include similar datasets, data preprocessing methods and evaluation metrics, more or less in a controlled manner. With this kind of comparison, it becomes easier to judge the strong points and weak points of each model, while also reducing any possible biases that may come from the experiments. Unlike many other studies that only look at classification accuracy alone, this research also brings in extra criteria, such as precision, recall, F1 score, confusion matrices, and classification reports.

One other key point worth highlighting about the study is its development of a workable workflow for automated classification of rice leaf diseases, which can be replicated and further developed by subsequent researchers. This includes dataset collection, image pre-processing, data augmentation, model training, transfer learning and performance assessment, among other pieces that fit together kind of naturally. Also, the study gives a clear, technically grounded explanation of why the transfer learning model underperformed in practice, and that outcome, it is itself useful and transferable for future practitioners who will be adapting similar pipelines .

1.10 Report Organization

The document is kind of divided into six main chapters that cover all aspects of the making, running, assessment, and inspection of the suggested rice leaf disease classification model. Chapter 1 kind of starts off the document by tackling the topic , through the

presentation of the application background, the importance of the problem, the machine learning side of the work , ideas behind transfer learning , the concept of explainable AI , research gap, problem statement, research objectives, research questions, and the key contributions of the project. Chapter 1 basically gives the motivation and foundation for the study, and it also outlines its broad direction.

Chapter 2 then provides a more detailed review of related literature that addresses rice and plant leaf disease detection, deep learning, convolutional neural networks, transfer learning, and practical uses of computer vision for agricultural monitoring. The chapter assesses earlier studies, it brings out the gaps in the literature, and it points to chances for more exploration. In addition , a matrix for comparative literature review is also provided in this chapter.

The research methodology used in this study is explained in Chapter 3. Here , there is an account of the data used, data pre-processing, image augmentation methods, model development, transfer learning, training, evaluation metrics, and the experiment set-up. This chapter includes adequate details to enable replication of the proposed system along with the experiments.

The fourth chapter gives the results from the experiments that were done using the developed models, organized around the five research questions stated earlier. This chapter includes dataset statistics, classification metrics, a confusion matrix, a classification report, and a comparison between the Custom CNN model and the EfficientNetB0 model. Figures, and tables are also used so the results can be shown visually, not only described.

In Chapter 5, the findings are presented, and the interpretation of those findings is made with respect to the research questions and earlier literature. The advantages plus disadvantages of the proposed framework are analyzed. Also, the underlying reason behind the transfer learning model's underperformance is diagnosed and the practical implications of the results are emphasized. Finally , further directions for research are mentioned, with the aim to improve how effective automated rice disease detection systems can be.

Lastly, Chapter 6 presents the conclusions of the research. In Chapter 6, the main findings, contributions, and overall outcomes are summarized again. It also underlines the research objectives and the significance of the findings, plus why deep learning–based rice disease detection systems matter for agricultural monitoring and food security.

2 Literature Review

2.1 Existing Rice and Plant Leaf Disease Detection Studies

For the classification of rice and plant leaf diseases, computer vision has been something people work on for more than ten years, more or less. The early attempts kinda leaned on

classical image processing pipelines , then they paired those with more traditional machine learning classifiers like Support Vector Machines (SVMs). In that setup, researchers used crafted cues— for instance color and texture descriptors , stuff you can "design" by hand. Those methods could still reach decent results on smaller, controlled datasets, yet they usually fell short when they had to deal with field images , because those come with changing light conditions, busy backgrounds, and different camera angles. So, in practice, the models didn't generalize well.

A systematic review focused on deep learning for rice disease diagnosis mentions that hybrid strategies—some mix of classical and deep methods, like SVM with K-means clustering—reported training accuracies near 95% on more restricted datasets. At the same time, CNN approaches that use transfer learning have typically surpassed plain CNNs when the datasets are small and collected from real fields. For example, on a four-class rice leaf disease set containing 5,932 field images, a configuration where a CNN with transfer learning was used together with an SVM performed better than using the CNN alone. Meanwhile, other early work tried VGG16 with transfer learning on a small collection of rice and wheat leaf images and sometimes showed extremely high accuracy values, but those results were often obtained on datasets that are several orders of magnitude smaller than the one in this study, so it's hard to compare them directly, if you think about it.

Taken together, these papers suggest that deep learning is often much more useful than conventional visual inspection methods because it provides automatic feature extraction, and it can push classification accuracy to higher levels , even when the image conditions are not perfectly controlled.

2.2 CNN-Based Rice Disease Detection

Custom CNN architectures designed specifically for rice disease classification have been pretty studied, I mean, like a lot. Some work built an automated diagnostic system for rice leaf diseases using a big dataset of images covering six common rice diseases: bacterial stripe, false smut, leaf blast, neck blast, sheath blight , and brown spot. They showed that purpose-built CNN pipelines can scale to broader and more diverse rice disease datasets than earlier studies ever managed. Other research also looked at lightweight custom architectures that are tuned for rice disease classification, such as reduced MobileNet variants with depthwise separable convolutions, enhanced VGGNet variants , and compact CNN architectures that report accuracies above 90% while staying smaller in size. That kind of result suggests that when compute is a concern, carefully crafted, dataset-specific designs can be competitive with, or even sometimes preferable to, bigger pretrained networks.

Separately, a hybrid deep learning study aiming at paddy leaf disease identification

reported an accuracy of 98.86%. Their workflow was kind of structured, you know image acquisition, preprocessing, feature extraction, feature selection, and classification, all in sequence, mostly. Related work focused more narrowly on automated detection of brown spot disease in paddy leaves. In that case, across several CNN-based architectures evaluated with sensitivity, specificity, precision, and F1-score , a VGG-19 model came out on top, with the best accuracy at 93.0%.

Overall, these findings illustrate that CNN-based rice disease detection often reaches high reported accuracy when the dataset and the experimental protocol are aligned with the architecture being used. It also reinforces the importance of matching the architecture and the data pipeline, which is basically the main issue addressed in this study's work.

2.3 Transfer Learning Approaches

Transfer learning is one of those increasingly popular methods in deep learning research, mainly because it helps researchers take the know-how gathered from huge datasets and then, kind of reuse it, for other applications. Instead of training a deep neural network entirely from zero, transfer learning leans on pre-trained models that already learned generic features for visual recognition, often by looking at millions of images. Then, this existing experience can be moved over to a particular problem by swapping out and fine-tuning the classification layers that sit near the end of the network layout. In practice, this approach really cuts down on time, computing power, and also the amount of training data that is needed for development. Transfer learning has shown big results in computer vision too, especially across image classification, object detection, medical imaging, remote sensing, and agricultural monitoring.

A number of papers have been published about applying transfer learning frameworks to rice and paddy disease classification. When benchmarking work was done using the original Paddy Doctor dataset release, it was reported that a ResNet34-based setup reached a 97.50% F1-score on a 16,225-image annotated dataset, so it basically showed that transfer learning can work very well on this kind of rice disease information, provided the configuration is done properly. Similarly, another study trained a pretrained EfficientNetB3 convolutional neural network for identifying and categorizing paddy leaf ailments using the Paddy Doctor dataset, and yet another EfficientNet-based investigation—focused on paddy leaf disease classification with compound scaling and swish activation—reported that EfficientNetB4 hit an accuracy of 100%, beating traditional visual inspection methods. In the same line, VGG16 came close, with 99% accuracy.

However, not all studies report uniformly strong results for transfer learning on paddy-related classification tasks. A recent comparative benchmarking study, evaluating lightweight and deep CNN models across multiple paddy datasets, found that transfer learning was kind of hit-or-miss on a paddy variety dataset: while EfficientNetB0 and

ResNet18 models initialized with pretrained weights showed smoother loss decay, and steadier convergence than their scratch-trained counterparts . still, their final classification performance turned out lower, and the authors link that to the chance that the features learned from large-scale natural image datasets may not really capture those subtle visual differences that are needed for fine-grained agricultural classification tasks. This is especially relevant to the present study, because it shows, using a benchmark published separately, that transfer learning can trail a from scratch model trained on rice or paddy imagery , and that situation isn't unprecedented, it can even happen without any implementation error, giving extra context to the outcome for EfficientNetB0 reported in this study.

2.4 Comparative Literature Review Matrix

Table 1 presents a sort of comparative summary of the most relevant studies reviewed through the literature. the matrix, it kind of highlights the datasets, the methodologies, the performance outcomes, and also the limitations that were reported by previous researchers. this comparison gives a clear enough overview of current research trends and it points to opportunities, for further investigation in rice leaf disease classification, which is kind of ongoing and still a bit open ended.

Table 1: Comparative Literature Review Matrix

Source	Dataset	Methodology	Performance	Limitations
Petchiammal et al. (Paddy Doctor benchmark)	Paddy Doctor (16,225 images)	ResNet34 transfer learning	97.50% F1-score	Benchmark only on a single architecture family
Padhi et al.	Paddy leaf images	EfficientNetB4, compound scaling	100% accuracy	Possible overfitting risk at reported ceiling accuracy
Rice leaf disease ensemble study	18,563 images, 6 diseases	Ensemble CNN models	High accuracy (dataset-dependent)	Large curated dataset; ensemble adds inference cost
Hybrid deep learning framework	Paddy leaf dataset	Multi-stage hybrid CNN pipeline	98.86% accuracy	Complex multi-stage pipeline, harder to reproduce
VGG-19 brown spot study	Paddy leaf images	VGG-19 transfer learning	93.0% accuracy	Focused on a single disease category
Lightweight/Deep CNN comparison (PaddyVariety)	Paddy variety dataset	EfficientNetB0 / ResNet18, scratch vs. transfer	Transfer learning underperformed scratch-trained models	Demonstrates transfer learning is not always superior on paddy imagery
This study	Paddy Doctor (10,407 images)	Custom CNN vs. EfficientNetB0 transfer learning	Custom CNN: 76.2%; EfficientNetB0: 16.9% (diagnosed preprocessing mismatch)	Single train/test split; transfer learning result requires correction and re-evaluation

The reviewed studies kind of show that deep learning approaches, like custom CNNs and transfer learning setups, work pretty well for identifying rice and paddy leaf diseases, yet the outcomes seem very touchy, like they depend on the dataset itself, plus the exact preprocessing decisions and even the architecture specific way things get implemented. So, these findings kinda justify why we should run a comparative check between a Custom CNN and the EfficientNetB0 model, especially using the publicly available Paddy Doctor dataset.

2.5 Chapter Summary

From the literature that was discussed in this chapter, it becomes kind of obvious that a lot of progress has been made in catching rice and plant leaf diseases, mostly using deep learning ideas together with computer vision. At the same time, the old manual checking

ways are slowly giving way to more automatic and efficient processes that rely on image analysis. Even so, there are many studies in this area, where CNNs, transfer learning methods , and hybrid deep learning algorithms are used, specifically for identifying plant disease. Based on what is already shown in the literature, it also looks like deep learning models can actually learn rather complex visual patterns linked with rice diseases and then sort them in a fairly effective manner.

Transfer learning approaches have also gained quite a bit of attention in agricultural monitoring, and honestly they seem to be more and more relevant. In addition, architectures like ResNet , VGG, MobileNet and EfficientNet have demonstrated strong performance, because they can predict well while also reducing the training time and saving computational effort. Still, these results can not always be matched side by side in a clean way, because many researchers work with different datasets, use different evaluation setups, and report different metrics. Also, quite often, the focus stays mostly on the overall accuracy, and then other key parameters get ignored, like class-level performance, model interpretability, or implementation details. So, it seems that more comparative work should be done, in order to see the different approaches more clearly, especially in situations where transfer learning doesn't meet expectations.

Taking into account the research gaps outlined above, this paper explores a kind of possibility where a Custom Convolutional Neural Network plus an EfficientNetB0 transfer learning setup can be used for classifying rice leaf diseases. These two approaches will be checked in a single experiment, and both models will be trained on the same datasets, while using the same preprocessing and evaluation techniques. So the results are made comparable, and it becomes easier to see the advantages or disadvantages of each model, even if that is not always straightforward. The next chapter describes the research methodology used to conduct the study.

3 Research Methodology

3.1 Research Design

In this study, a quantitive experimental design methodology was used a bit, to see how well deep learning algorithms work for automatic rice leaf disease classification. Basically, the idea was to build image classification models that could sort the given images into ten categories, nine disease types plus one healthy normal. Supervised learning was used because there was a labeled dataset available, so it could be employed for training and for testing as well. The research setup involved creating both a Custom Convolutional Neural Network, and a transfer learning model built on the EfficientNetB0 architecture. These two approaches were trained and evaluated using the exact same dataset, with the same preprocessing steps and performance metrics, just so the comparison stays fair.

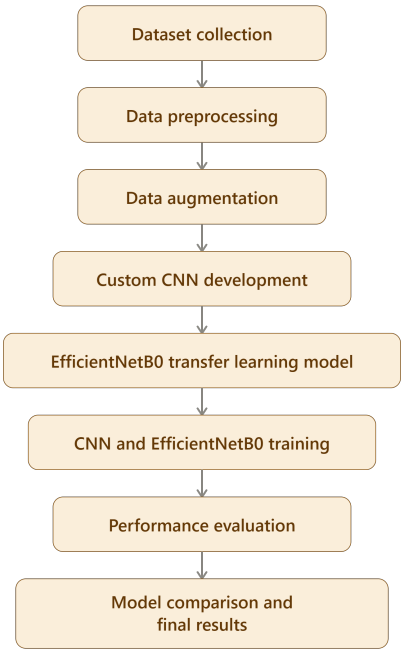

Figure 2: Overall Research Workflow

Overall, the procedure went something like dataset preparation, image preprocessing, data augmentation, model creation, model training, then model performance assessment, and finally model comparison. After that, the experiment outcomes were judged with multiple evaluation metrics such as accuracy, precision, recall, F1-Score, a confusion matrix, and classification reports.

3.2 Dataset Description

In total, the dataset contains 10,407 images. With stratified sampling applied, the dataset was divided into 8,326 training images (around 80%), 1,041 validation images (about 10%), and another 1,041 testing images (10%). These splits were used across the training stage, model tuning, and the last evaluation step. So, having training, validation, and testing subsets in place gives a more grounded experimental design and, it also helps reduce evaluation bias. Before the model was trained, every image was resized to a uniform 224 x 224 pixels, so that it fits both the Custom CNN design and the EfficientNetB0 transfer learning approach.

To make the dataset traits easier to grasp, Table 2 gives a summary of the main properties of the dataset applied in this work.

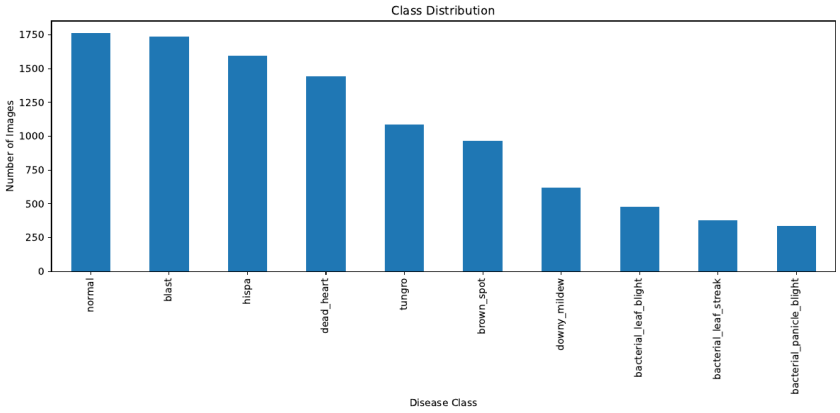

Figure 3: Class Distribution Across the Ten Rice Leaf Disease Categories

Table 2: Dataset Description

Dataset Name	Paddy Doctor: Paddy Disease Classification
Source	Kaggle
Application Domain	Rice Leaf Disease Detection, Precision Agriculture
Total Images	10,407
Training Images	8,326 (approx.)
Validation Images	1,041 (approx.)
Testing Images	1,041
Number of Classes	10
Class Labels	normal, blast, hispa, dead_heart, tungro, brown_spot, downy_mildew, ba
Input Resolution	224×224
Framework	TensorFlow/Keras
Development Environment	Kaggle Notebook

Table 3 presents the class-wise image counts, and Figure 3 visualizes this same distribution.

Table 3: Class Distribution

Class	**Number of Images**
normal	1764
blast	1738
hispa	1594
dead_heart	1442
tungro	1088
brown_spot	965
downy_mildew	620
bacterial_leaf_blight	479
bacterial_leaf_streak	380
bacterial_panicle_blight	337
Total	**10,407**

As shown in Table 3 and Figure 3, the dataset exhibits moderate class imbalance:

the largest class (normal, 1,764 images) contains roughly 5.2 times as many images as the smallest class (bacterial_panicle_blight, 337 images). This imbalance is an important factor in interpreting the per-class results presented in Chapter 4.

3.3 Data Preparation

The process of preparing the data is kind of crucial for building effective deep learning models, because the quality and consistency of the input data kinda decide how well the classification results will come out. The images went through a set of preprocessing procedures before actually building the deep learning models, so that the images can have the same size. In particular, the starting dataset contained images with different resolutions and varying visual characteristics , so each image was resized to reach 224 $\times$ 224 pixels. This size was picked , because it works well for both the Custom CNN model and the EfficientNetB0 model that were used in the study.

Normalization was also applied, as part of that preprocessing stage. More specifically, the pixel intensities were scaled down from 0–255 to 0–1, by dividing every pixel by 255. With this kind of normalization , the training tends to be numerically stable and the optimization step can converge faster. Also, beyond the points above, there were additional data augmentation techniques used, to increase the variety inside the training set. The augmentation strategy included random rotations (up to 20 degrees), width and height shifting (up to 20%), zooming (up to 20%), and horizontal flipping. The validation and test sets were not augmented at all; they only got rescaled, so that the evaluation would reflect performance on more real-world, unaltered images.

The data were partitioned into training, validation, and testing sub sets, which was meant to make sure there was an appropriate framework where the model accuracy could be checked, and it also relied on stratified sampling plus a fixed random seed 42 so that the class proportions stay consistent across all three. The training portion itself contained 8,326 images and it was what the model learned from during training, basically. Then there was a validation subset with 1,041 images, and that part was used to monitor the performance while training, as well as to help with the early stopping calls, kinda guiding when to stop. Finally, the testing subset also had 1,041 images and it was reserved for the assessment of the finished model. Overall, this whole split procedure helped ensure that the metrics were gathered from data the model hadn't seen before, so the evaluation stayed more fair, in terms of generalization ability, for the models.

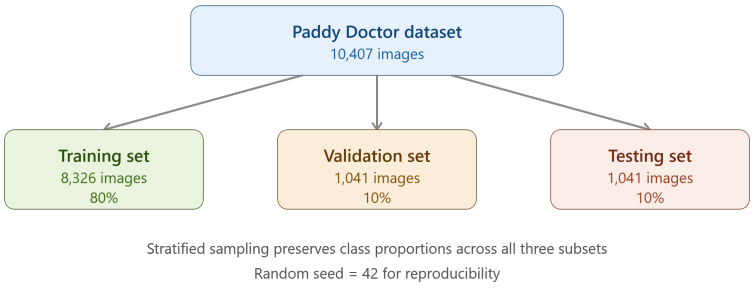

Figure 4: Dataset Partitioning Strategy

3.4 Model Development

The whole process around building a model architecture, aiming at the ability to classify rice leaf images into ten disease categories was carried out in this study. Two directions were tried, kind of side by side. The first one was to create a Custom Convolutional Neural Network (CNN) setup that was tuned specifically for rice leaf disease classification. The other direction was about transfer learning , and it was implemented on top of the EfficientNetB0 backbone, a rather common and well known deep learning architecture pretrained on ImageNet.

Using both methods in the same experimental conditions meant the performances could be compared without lots of extra changes or "hidden" differences.

The Custom CNN was designed with four convolutional blocks then fully connected layers. The input images were fed in with shape $224 \times 224 \times 3$, and then convolution operations were applied to form a kind of visual ladder of features. The very first convolutional layer used 32 filters with a 3×3 kernel, along with a ReLU activation, and it started extracting low level details. In later convolutional stages , the filter count went up to 64 , then 128 , and finally 256, so the network could learn more complicated visual patterns over time. After each convolution stage, max pooling was added, to shrink the spatial size and also keep the computations manageable.

Once the features were captured, the pipeline sent them to a dense layer with 256 neurons , then to a dropout layer with 0.5 probability. After that, a Softmax output layer containing ten neurons produced the class probabilities for the ten rice leaf disease categories.

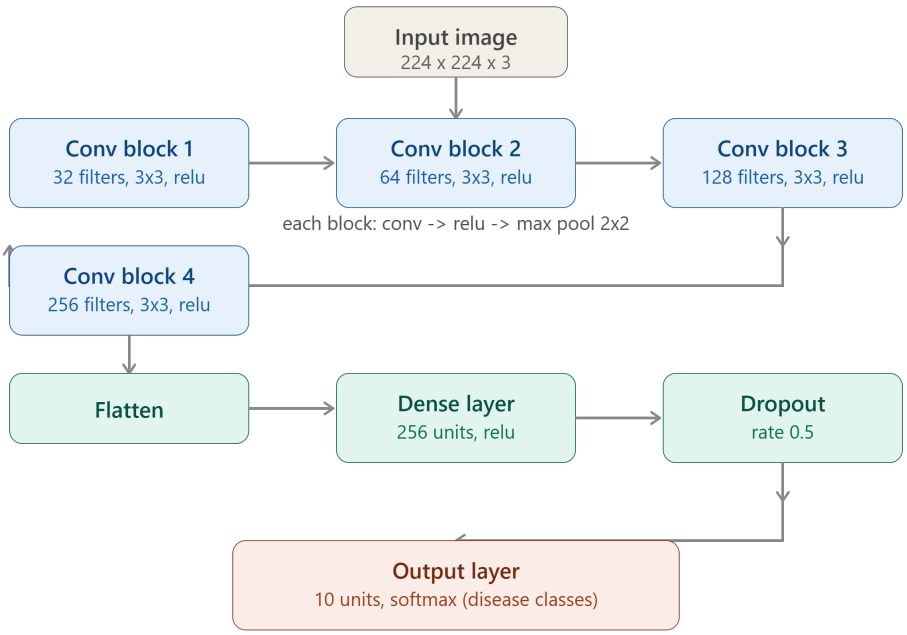

Figure 5: Custom CNN Architecture for Rice Leaf Disease Classification

The Custom CNN architecture has four conv blocks, with 32, 64, 128, and 256 filters each, kind of in that order. In every convolutional step, there is max pooling right after it, to make feature extraction more efficient , and generally easier. After that, the learned features get flattened and then sent into a dense layer, this one has 256 neurons , plus a dropout layer, before the final classification part with a Softmax output. A full layer by layer parameter breakdown matching this setup is listed in Table 4.

Table 4: Custom CNN Layer Summary

Layer	**Output Shape**	**Parameters**
Conv2D (32 filters)	$222 \times 222 \times 32$	896
MaxPooling2D	$111 \times 111 \times 32$	0
Conv2D (64 filters)	$109 \times 109 \times 64$	18,496
MaxPooling2D	$54 \times 54 \times 64$	0
Conv2D (128 filters)	$52 \times 52 \times 128$	73,856
MaxPooling2D	$26 \times 26 \times 128$	0
Conv2D (256 filters)	$24 \times 24 \times 256$	295,168
MaxPooling2D	$12 \times 12 \times 256$	0
Flatten	36,864	0
Dense (256 units)	256	9,437,440
Dropout (0.5)	256	0
Dense / Softmax (10 units)	10	2,570
Total Parameters		**9,828,426**

Other than the Custom CNN model, a transfer learning strategy using EfficientNetB0

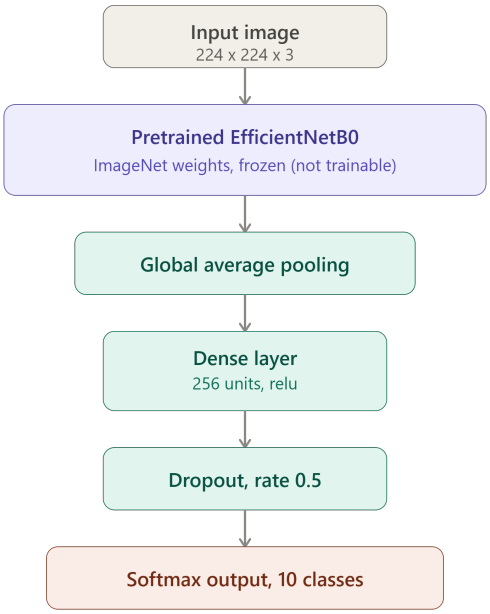

Figure 6: EfficientNetB0 Transfer Learning Architecture

as the base architecture was used in the study, to see how well a pretrained deep learning setup could work for rice leaf disease classification. EfficientNetB0 is basically a convolutional design that applies compound scaling, so the balance between depth, width, and resolution stays more or less in check, and it is often reported to reach solid accuracy on ImageNet with a relatively small parameter count. To make use of the visual knowledge the network learned earlier, specifically during its ImageNet training phase, the ImageNet weights were loaded to initialize the EfficientNetB0 model. After that, the initial classification head of EfficientNetB0 was removed, and it was replaced with a Global Average Pooling operation, then a dense stage with 256 neurons, followed by a dropout component, and finally a Softmax output layer for ten classes. During training, the EfficientNetB0 backbone stayed frozen , while only the newly attached classification layers were optimized.

So the EfficientNetB0 transfer learning model uses those pretrained ImageNet weights as a kind of feature extractor, yeah. Then the initial classification head is taken away and swapped out for a Global Average Pooling block, followed by a dense layer with 256 neurons, plus a dropout step and lastly the Softmax output. That final output handles ten-class rice leaf disease classification.

3.5 Training Procedure

The training process was sort of aimed at making sure learning stayed as good as possible, while also lowering the chance of overfitting and performance degrading over time. Training for both the Custom CNN and EfficientNetB0 models was carried out with the

Adam optimizer, which is popular because it has adaptive learning behavior and tends to converge quickly, so yeah. Categorical cross-entropy loss was selected to train the models since the current task had ten mutually exclusive output classes, nothing overlap there. For evaluation during training, accuracy was used as the metric to keep an eye on how the models were doing, mostly because it is easy to interpret, pretty straightforward. Everything was trained using TensorFlow and Keras deep learning libraries inside the Kaggle Notebook platform.

The Custom CNN was trained in mini batches of 32 images and had been set up so training would not go beyond 20 training epochs, with early stopping (it was monitoring validation loss, patience of 5 epochs, and restoring the best weights) used to halt training once validation performance stopped getting better. Each training epoch also involved going through the dataset, updating the network parameters using backpropagation, and then checking the model performance on the validation dataset , just to be sure. Using the validation dataset let us follow how well the model generalizes, and also to spot potential overfitting. Throughout the whole training run, the Custom CNN kept improving at extracting discriminative visual cues for the ten disease classes. Dropout together with data augmentation helped the model stay more stable and boosted performance.

However, the EfficientNetB0 transfer learning model did go through a slightly modified training process. In particular, the pretrained convolutional layers of EfficientNetB0 were frozen, so the ImageNet features stay basically untouched. It is just the extra classification layers that were actually trained using the rice leaf disease data. The max number of epochs for this setup was set to 10 , with the same batch size and early stopping configuration as the Custom CNN. So in practice, these train ing processes let both models get optimized under more or less similar conditions, and made it possible to do an objective comparison of how well they classify rice leaf disease images.

3.6 Evaluation Framework

The usefulness of the built models for rice leaf disease classification was estimated by employing various performance metrics, so the evaluation of predictive capabilities is more or less complete. Relying on classification accuracy alone can sometimes give an incomplete picture, especially when some classes are predicted in a mixed way, which is really relevant considering the class imbalance shown in Table 3. Because of that the present research also used other performance measures like precision recall, F1-score, confusion matrices, and classification reports. Together these metrics offer a more detailed view of what the models do for each class, not just overall. Therefore the estimation framework was selected to consider not only a whole-picture analysis, but also a class-wise reading.

Accuracy functioned as the primary yardstick for the classification quality and it is

defined as the ratio of images that were correctly classified to the total number of samples under consideration. Accuracy is a straightforward performance signal and it is widely used for judging image classifiers. Even so, accuracy by itself does not tell you how well each individual class is recognized. To address that, precision, recall, and the F1-score were included in the set of criteria. Precision describes the percentage of truly correct samples that belong to a given class out of all samples that the model assigned to that same class. Recall, in contrast, measures the classifier's ability to retrieve all samples that are actually relevant for a particular class.

Confusion matrices were put together to kind of make sense of the classification outcomes and, see where things went wrong, between the ten rice leaf disease categories. They basically give a lot more detail than you'd think at first about what got labeled right and what got misclassified. Also, classification reports were made to show the precision, recall, F1-score, and support for each category. In practice these reports made it easier to compare the two models, Custom CNN and EfficientNetB0, in a deeper way than accuracy alone really could. By stitching together accuracy plus precision, recall, F1-score, the confusion matrix , and those classification reports, we had a solid way to assess the models and answer the research questions that were listed in the study.

3.7 Explainable AI Methodology

Explainable Artificial Intelligence (XAI) has become kinda important in AI research because it helps make deep learning models more transparent and more trust worthy. Deep neural networks can still classify images pretty well, but the way they actually reach their decisions is not easy to interpret at all. And since using deep learning in real life comes with some risks, like potential classification errors that might lead to real economic consequences for farmers, it becomes helpful to be able to somehow understand why a model made a specific choice. Because of that, many newer studies take an explainability of deep learning models angle, to look at how the algorithm spots important visual cues in the first place. Some well known methods people use are Grad-CAM and SHAP.

In this research study, two models were designed and then compared: a Custom CNN and an EfficientNetB0 transfer learning model. So, in terms of what this project actually covered during implementation, the main attention was on dataset creation, the training of both models, and the evaluation of performance using quantitative metrics. Grad-CAM and SHAP were considered from the beginning, since they can provide explanations about how the models come to their outputs. Still, they could not end up included in the experiments that were carried out now, due to limited project scope and time pressure, in a way. This limitation should not affect the classification performance results that were achieved, however it does mean the models' decision making pathways cannot be directly verified.

The future research may go toward applying explainability in a slightly different manner, to further enhance the proposed approach for detecting rice leaf diseases by developing more dependable models. In particular, using Grad-CAM visualization could point out those areas of the images that, like pretty decisively, influence the prediction of each disease class, while SHAP might give even more clarity regarding what features matter. This way, people could verify whether their models are actually focusing on significant disease related regions instead of, you know, relying on irrelevant background patterns. At the same time, explainability would help expose potential biases, which makes debugging easier and supports more sensible operational decisions when deploying the model.

3.8 Robustness Analysis

Robustness of a deep learning model can be sort of understood as the ability of the model to keep giving reliable results when the input dataset changes, or when it's in an unknown environment. Robustness becomes really important for rice leaf disease detection, because you might end up with images captured under varied lighting conditions, different backgrounds , unusual camera angles, and also at different disease progression stages. If a model performs well on its training data, yet can't actually generalize to new images then, honestly, it becomes kinda impractical. So during the design of the proposed system, a few steps were taken to boost robustness and also reduce overfitting. Those steps included data augmentation , dropout regularization, monitoring based on validation, and finally testing on a separate independent test dataset.

In this research, data augmentation was one of the main methods used for robustness enhancement. The training images went through random transformations like rotation, shifting, zooming, and horizontal flipping , and the idea was to make the model exposed to multiple "views" of the same general scenario. These transformations helped the model become more responsive to the different look of the images, rather than sticking to one consistent pattern. In turn, the augmented training data supported the learning of features that generalize beyond the training set, instead of simply memorizing samples. This approach is already widely recognized as a solid way to improve the generalization ability of models in computer vision tasks. Therefore, the augmentation technique was a key piece in building a robust classification system for rice leaf diseases.

More approaches for making the system more robust were carried out on both the model architecture and the training side, if you will. For example the dropout layer was switched on with a drop rate set to 0.5 so as to reduce the risk of overfitting, basically by not giving too much influence to single neurons. In parallel, the validation dataset was monitored for the entire training run, to help judge the generalization capability of the model and also to flag overfitting early, before it gets too bad. On top of that, there was a separate testing dataset, containing images the model had never seen, which was

used to get a fairly unbiased picture of how well the model performs in practice. Still, it should be mentioned that these robustness steps mainly concern the Custom CNN. As mentioned in Chapter 5, the EfficientNetB0's weaker results here are not really what people usually mean by an overfitting or robustness failure. Instead, it seems to come from a preprocessing mismatch, meaning the frozen backbone received inputs that were not aligned with what it expects, and so the backbone did not behave as intended.

3.9 Software, Hardware and Reproducibility

The suggested rice leaf disease classification framework was executed using the Python language, and TensorFlow/Keras deep learning libraries, sort of, as the main stack. The choice of these tools was made because they have solid strengths for neural network structuring, transfer learning , image processing, and model evaluation. Other supporting libraries included NumPy, Pandas, Matplotlib, Seaborn, and Scikit-learn, which were utilized for handling the data, visualizing, and then assessing how well the experiment performed. Everything, more or less, ran inside the Kaggle Notebook development environment, so we got GPU-accelerated compute and a stable environment that is shareable for both models.

The Custom CNN model and the EfficientNetB0 model were trained under the same computational conditions, in order to keep the comparison reasonable. A consistent pipeline was used for preprocessing, hyperparameter decisions, and evaluation across all experiments. This single-style method restrained the influence of outside factors , and it improved the credibility of the results shown.

Reproducibility matters a lot in scientific work since it allows other researchers to check the conclusions and extend them. For that reason, the full implementation of the project was written down and kept in a GitHub repository plus a Kaggle notebook that could be viewed publicly. The repository also contains the required pieces, like the source code, details about the dataset, the model settings, and the evaluation process notes, all so the study can be repeated , more or less, without guesswork.

3.10 Ethical and Practical Considerations

Development and the implementation of AI solutions for crop disease monitoring kinda have to be handled while also respecting ethical, practical, and social concerns. On one side, automated rice disease detection systems seem useful, mainly because they can give quicker diagnoses and make monitoring more accessible, but on the other side, erroneous predictions can bring negative consequences. If there are false negatives, the treatment may get delayed, and the chance of crop loss rises. If there are false positives, it might lead to unnecessary pesticide application, or basically wasting resources. So deep learning

models should be tested properly , and their limitations should be explained clearly before anyone deploys them in everyday practice.

For this study, the dataset was taken from a freely accessible source, and it included image data that was intended for research and educational analysis. Personal information wasn't part of the dataset, and no human subjects were involved in the research, so privacy worries are fairly minimal. Responsible use of machine learning here also means that we need to share transparent details about the model's real performance, not just present it as fully dependable. That includes the weaker, under-performing transfer learning result that is discussed in this report , rather than overstating how reliable the whole system is.

4 Results and Analysis

4.1 Dataset Statistics

In this experiment, the dataset used comprised 10,407 images across ten rice leaf categories; nine disease classes and one healthy normal class. The dataset was split into three subsets , i.e., training, validation and testing sets, to allow for model development and evaluation (it was kind of standard but still). The number of images in the training subset was 8,326, whereas 1,041 images made up the validation subset. On the other hand, there was a testing portion of 1,041 images, used for final evaluation of the model performance. All images were resized to 224 x 224 pixels and preprocessed along with augmented versions before training. The dataset seemed sufficiently varied to test how well deep learning models could tell apart those ten rice leaf disease categories.

The results below are arranged around the five research questions introduced in Section 1.8.

Question 1: Can a Custom CNN accurately classify rice leaf images into the correct disease category?

The Custom CNN got pretty good results for the image classification task tied to rice leaf diseases. Basically the neural network model was trained for as much as 20 epochs, using the Adam optimizer and categorical cross-entropy loss, and then early stopping was set based on the validation loss. In the training part the net kept improving for accuracy, and the validation stayed pretty steady, so by the last epoch it reached around 68% training accuracy and about 75% validation accuracy. Also, adding dropout and data augmentation, helped with better generalization, and it made overfitting a bit less likely, you know, that common issue. Next the process moved to evaluation on the testing dataset, which includes new images it had not seen. The final accuracy of the Custom CNN came out to 76.18%. That suggests the method is fairly efficient, especially for

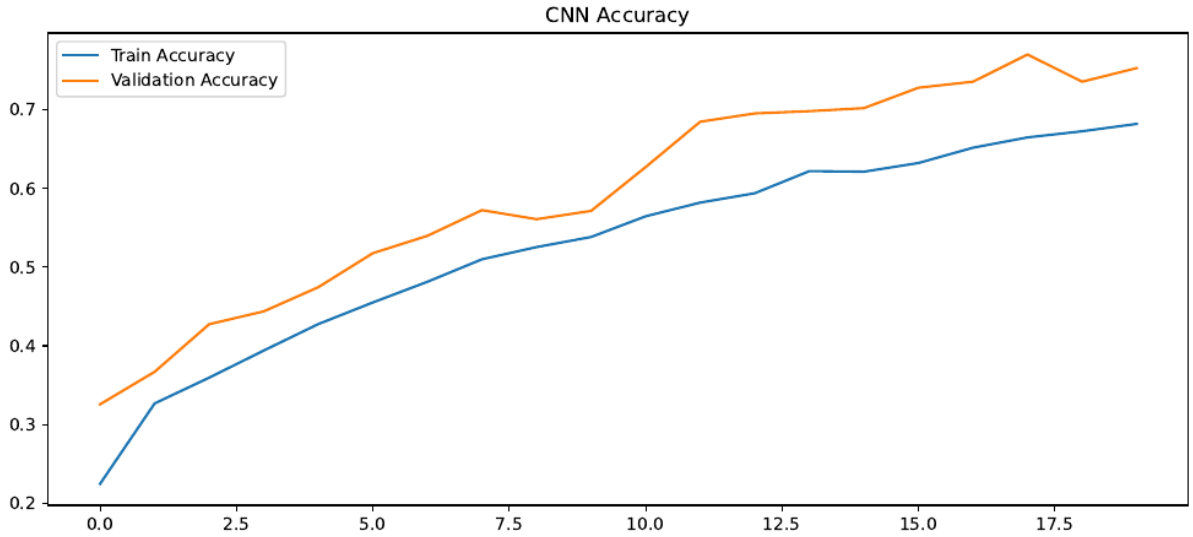

Figure 7: Custom CNN Training and Validation Accuracy Across Epochs

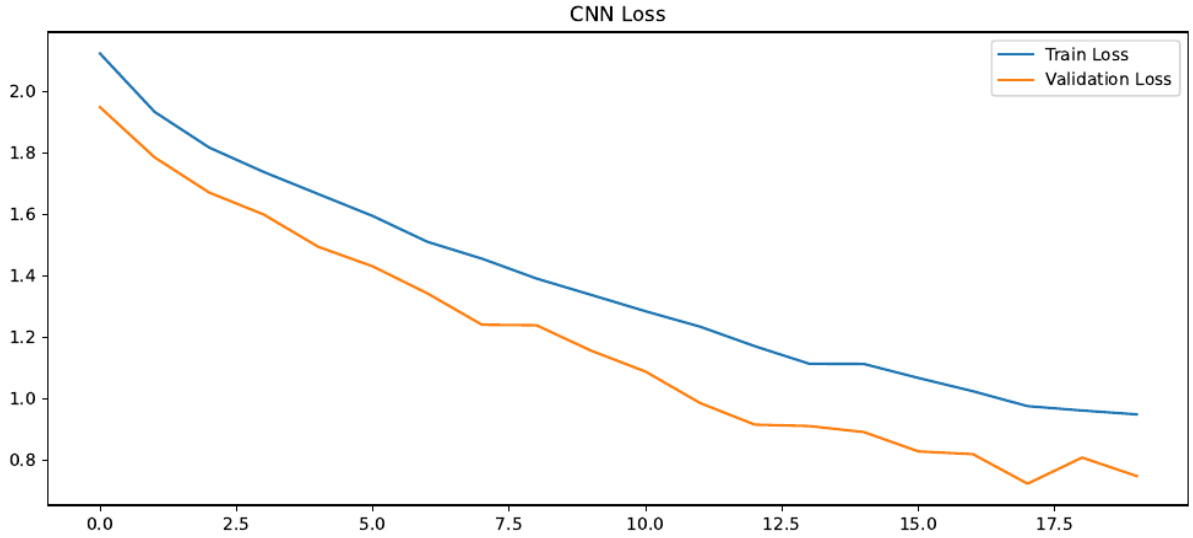

Figure 8: Custom CNN Training and Validation Loss Across Epochs

recognizing the ten rice leaf disease categories.

Table 5: Custom CNN Performance Summary

Metric	**Value**
Model	Custom CNN
Test Accuracy	76.18%
Macro F1-score	0.714
Weighted F1-score	0.755

The confusion matrix gives a kind of full picture about how well the Custom CNN model works for the ten classes were looking at. In Figure 9 it kind of shows, how the test phase separates correctly predicted samples from the incorrectly predicted ones. Overall,

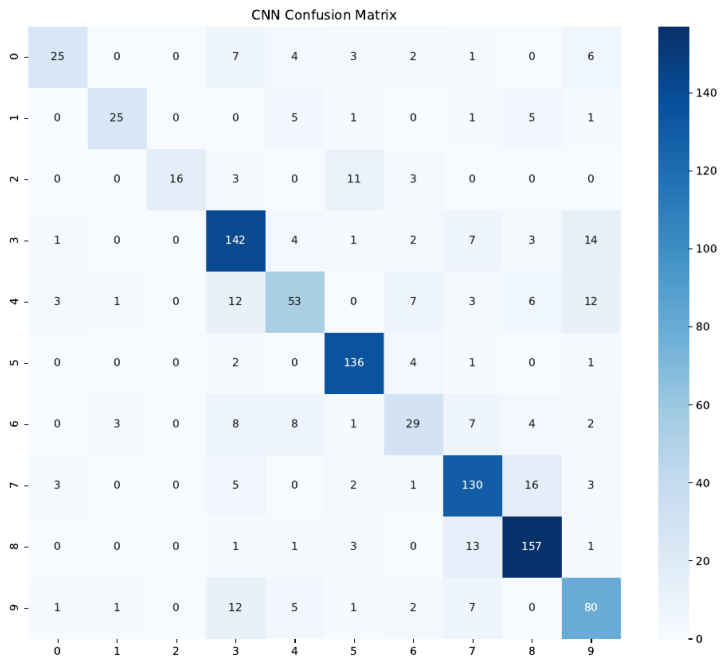

Figure 9: Confusion Matrix of the Custom CNN Model

most of the data managed to get classified right, you can see it from the strong values on the main diagonal of the confusion matrix. In particular the dead_heart , normal and hispa classes seem really consistent, each one comes in above 89% recall, so yeah they're pretty reliable in that sense.

Table 6 presents the per-class precision, recall, and F1-score for the Custom CNN on the test set. The strongest categories were dead_heart (F1 = 0.898), normal (F1 = 0.856), and hispa (F1 = 0.788), all of which had either a large number of training samples or visually distinctive symptoms.

Table 6: Custom CNN Classification Report (Test Set)

Class	**Precision**	**Recall**	**F1-Score**	**Support**
bacterial_leaf_blight	0.758	0.521	0.617	48
bacterial_leaf_streak	0.833	0.658	0.735	38
bacterial_panicle_blight	1.000	0.485	0.653	33
blast	0.740	0.816	0.776	174
brown_spot	0.663	0.546	0.599	97
dead_heart	0.855	0.944	0.898	144
downy_mildew	0.580	0.468	0.518	62
hispa	0.765	0.813	0.788	160
normal	0.822	0.892	0.856	176
tungro	0.667	0.734	0.699	109
Accuracy			**0.762**	1041
Macro avg	0.768	0.688	0.714	1041
Weighted avg	0.762	0.762	0.755	1041

Interesting Observation: Yeah, a Custom CNN trained from scratch can take rice

leaf images and point them to the right disease category with pretty solid accuracy. In our case, the test accuracy lands at 76.2% and the F1-scores stay above 0.6 for eight out of the ten classes so it looks like the network truly picked up discriminative visual cues. That idea is reinforced by the confusion matrix, because all ten classes show up along the diagonal, like it's lining things up quite neatly in Figure 9.

Question 2: How does the performance of EfficientNetB0 compare with the Custom CNN?

So, the EfficientNetB0 transfer learning network was used, kinda to see if a pretrained deep learning model could boost image classification results for rice leaf disease images. The pretrained EfficientNetB0 weights that came from ImageNet were loaded into the model, and the original classification layer was swapped out for a new one, made for ten classes only. In other words during training, the pretrained convolutions stayed as they were, while the fresh classification part got tuned using the rice leaf disease dataset, more or less.

EfficientNetB0 then got trained with the very same preprocessing steps, the same train/validation/test splits, and the same evaluation setup that were already used for the Custom CNN model. Once the model finished and we tested it on the test dataset, the classification accuracy ended up being just 16.91%. That number is really close to the fraction of the test set covered by the single largest class, namely normal, with 176 out of 1,041 test images, or about 16.9%. This basically points to the idea that the model wasn't actually learning how to separate the classes, but it looked like it was just defaulting to one repeated guess.

Table 7: EfficientNetB0 Performance Summary

Metric	**Value**
Model	EfficientNetB0
Test Accuracy	16.91%

Figure **??** presents the EfficientNetB0 confusion matrix, which shows a markedly different and informative pattern from the Custom CNN: nearly every test image, regardless of its true class, was predicted as "normal."

Table 8 shows the EfficientNetB0 classification report, which confirms numerically what the confusion matrix shows visually: the model achieved perfect recall (1.00) on the normal class because it predicted that class for almost every input, but achieved zero precision and recall on every other class.

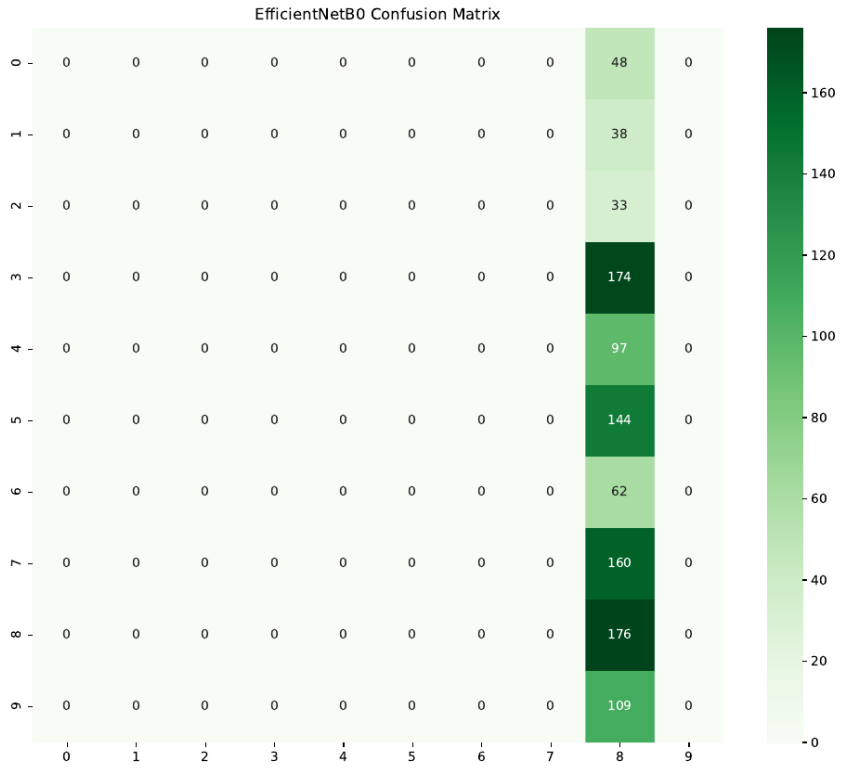

Figure 10: Confusion Matrix of the EfficientNetB0 Model, Showing Collapse to the Majority Class

Table 8: EfficientNetB0 Classification Report (Test Set)

Class	**Precision**	**Recall**	**F1-Score**	**Support**
normal	0.169	1.000	0.289	176
all other classes (9)	0.000	0.000	0.000	865
Accuracy			**0.169**	1041

The findings show that, as configured in this study, the Custom CNN model was far more suitable to the properties of the chosen dataset and could learn discriminative features from it, while EfficientNetB0's pretrained features were not effectively leveraged due to an implementation issue diagnosed in detail in Chapter 5. Table 9 and Figure **??** summarize the overall comparison.

Table 9: Comparison of Model Performance

Model	**Test Accuracy**
Custom CNN	76.18%
EfficientNetB0	16.91%

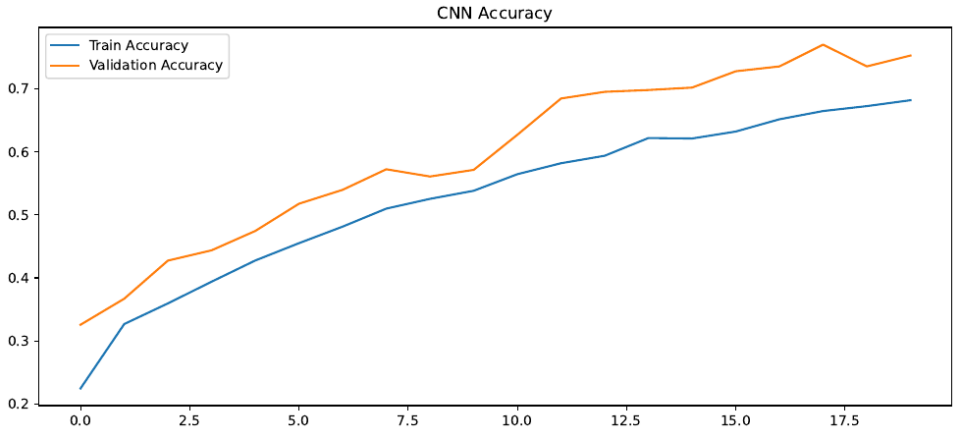

Figure 11: Accuracy Comparison Between Custom CNN and EfficientNetB0 Models

Interesting Observation: The Custom CNN ended up beating the EfficientNetB0 transfer learning model by a pretty huge margin, around 59 percentage points or so. But unlike a normal model comparison where both setups are truly configured the same way, and the gap means there's a real, measurable learning difference, this gap isn't really about what EfficientNetB0 can do. It was found, from the root cause work in Chapter 5, that most of it comes from a preprocessing mismatch for the EfficientNetB0 pipeline. Basically, the model is getting $[0, 1]$-scaled pixel inputs into a backbone that's looking for $[0, 255]$ inputs, and that kind of thing throws everything off. So it's not a property of the EfficientNetB0 architecture itself, not exactly, it's more about the input expectations not lining up.

Question 3: Which model provides better accuracy, precision, recall, and F1-score overall?

Based on every metric reported in Questions 1 and 2 above , the Custom CNN provided pretty substantially better performance than EfficientNetB0, especially in accuracy (76.2% vs. 16.9%), macro F1-score (0.714 vs. an effective macro F1 of around 0.029 for EfficientNetB0, since nine of its ten per-class F1-scores were 0.000). It also beat it on every individual class's precision and recall too. So as submitted , and evaluated in this study, the Custom CNN is therefore identified as the more effective model among the two evaluated approaches. This conclusion is accurate , with respect to the models exactly as configured and trained in this study (as emphasized in Question 2 and discussed further in Chapter 5) , but it should not be interpreted as a broad statement that custom CNNs are inherently superior to EfficientNetB0 for rice disease classification, given the preprocessing issue we found , which affected the transfer learning result.

Question 4: Which disease classes are most difficult to classify correctly, and why?

Refering back to Table 6, the downy_mildew and brown_spot classes were, like, the hardest for the Custom CNN to classify correctly , with F1-scores of 0.518 and 0.599 respectively , which are basically the two lowest across all ten classes. The bacterial_panicle_blight class also kinda stands out: it got perfect precision at 1.000, but the recall was only 0.485. So, the model often missed this category, even if it rarely did the wrong thing and tagged other samples as bacterial_panicle_blight.

Interesting Observation: These three classes share two characteristics that plausibly explain their difficulty. First, they are among the most underrepresented classes in the dataset (downy_mildew: 620 images, brown_spot: 965 images, bacterial_panicle_blight: 337 images, the smallest of all ten classes, as shown in Table 3), giving the model comparatively little training signal to learn their distinguishing features. Second, their visual symptoms overlap considerably with other disease categories such as blast and bacterial_leaf_blight, which the confusion matrix in Figure 9 confirms are common sources of misclassification for these classes.

Question 5: What are the strengths and limitations of the developed framework?

The main strength of this proposed framework is that it, basically manages to classify rice leaf images into ten disease categories, with solid accuracy (76.2%) using the Custom CNN, which is built entirely from a reproducible pipeline, covering preprocessing, augmentation, training , and multi metric evaluation. Another point is that this work did not just toss away or cover up the less performing EfficientNetB0 outcome; it actually worked through the issue and traced the result down to a specific preprocessing cause that can be corrected. That detail feels like something a lot of people can reuse, especially when someone is adapting an from-scratch CNN data pipeline to a pretrained architecture, and wants a kind of clearer path than "try and hope".

At the same time, there are limitations. First, the transfer learning experiment was misconfigured. Also there is the moderate class imbalance that is shown in Table 3. In addition, the evaluation relies on a single train/test split instead of doing k-fold cross-validation. And finally, there is no explainability analysis included, like Grad-CAM or SHAP, which means it is harder to interpret what the model attends to. These limitations and the related future work required to fix them are explained thoroughly in Chapter 5.

5 Discussion

5.1 Discussion of Results

The outcomes of the experiment done in this research kind of prove that deep learning approaches are effective in automated rice leaf disease classification, yes. The Custom Convolutional Neural Network managed to pick up suitable features, so it could tell apart ten different rice leaf disease classes. In particular, the Custom CNN reached a test accuracy of 76.18%, and the F1-scores stayed above 0.6 for eight out of ten classes, so overall the results look fairly balanced, even if not perfect. The most prominent categories were dead_heart, normal, and hispa, which implies that the visual traits tied to those states were sort of easier for the model to recognize. This may be because of a mix of larger sample numbers and symptoms that are more visually distinctive. On the other hand, downy_mildew and brown_spot were more troublesome, basically because they look similar to other diseases and they also had smaller sample sizes. Still, the achieved performance was reasonably strong, and it shows the proposed framework can discriminate between tricky, visually close disease conditions. The confusion matrix further suggests that most of the predictions sit along the diagonal , meaning a large portion of the classifications were actually correct.

In stark contrast, the EfficientNetB0 transfer learning setup hit only 16.91% accuracy. On its own, this outcome might make it look like the pretrained model was just not suited to this whole task. But the classification report plus the confusion matrix tell a much more exact tale, together they show the model kind of collapsed into predicting the majority class (normal) for nearly every image in the test set. It then lands perfect recall but it shows zero precision and recall for everything else, kind of everywhere. This overall pattern fits a scenario where the classification head basically learned to disregard its input, and then defaulted to the most frequent label instead. That behavior is a pretty strong signal that the trouble is likely in the input pipeline, rather than being a fundamental issue with the learning process itself.

5.2 Root Cause of the Transfer Learning Result

Most likely , this collapse comes from some preprocessing mismatch, between the data pipeline and the pretrained EfficientNetB0 backbone. In this study, the image data generator rescaled pixel values into the $[0, 1]$ range by dividing by 255, before handing them to both models. That part is the usual and correct normalization when you train a CNN from scratch. But the Keras version of EfficientNetB0 expects the raw pixel values to stay in the $[0, 255]$ range, because the network includes its own internal normalization layer, tuned to the precise statistics used during ImageNet pretraining. So when $[0, 1]$ scaled images get fed into a backbone that was expecting $[0, 255]$ inputs, the pretrained

convolutional filters end up seeing values way outside what they were trained on. That can lead to saturation , or at least it can mess with the activations as they move through the whole model.

Also , the EfficientNetB0 backbone was completely frozen. Meaning the system had no real chance to re-calibrate the early convolutional filters to offset the mismatch, during training. Only the new classification head was trainable, and if that head is receiving features that are basically unhelpful or corrupted coming from a frozen backbone with a distribution shift, it tends to choose the safest policy available. In practice that often becomes, always predicting the majority class. This lines up with the unusually low accuracy, and the particular collapse signature you see in the confusion matrix, along with the classification report too.

5.3 Research Question Analysis

This study was kind of guided by five research questions, about evaluating the effectiveness of deep learning techniques for classifying rice leaf disease, in a way that feels practical. The experimental findings from the Custom CNN and EfficientNetB0 models do give fairly clear answers to those same research questions. Also they help judge the usefulness of automated rice disease detection systems, like whether they can actually work outside the lab.

RQ1: Can a Custom Convolutional Neural Network (CNN) accurately classify rice leaf images into the correct disease category?

The results show that the Custom CNN did what it was supposed to, classifying rice leaf images pretty well, with test accuracy up to 76.18%. The model also got solid precision, recall,and F1-score values across most of the ten classes, not just one metric. Overall this suggests that a Custom CNN can learn discriminative look and feel features in a good way, and then tell apart the ten rice leaf disease categories fairly accurately, like it understands what matters visually, even when the patterns are subtle.

RQ2: How does the performance of a transfer learning model (EfficientNetB0) compare with a Custom CNN for rice leaf disease classification?

The comparative evaluation kinda showed that the Custom CNN substantially beat the EfficientNetB0 transfer learning model. Even though EfficientNetB0 only managed 16.91% accuracy, the Custom CNN got up to 76.18%. But tbh, that big gap may point to a diagnosed preprocessing error inside the EfficientNetB0 pipeline,so it probably isn't a clean or fair check of what the architecture could do for this job.

RQ3: Which model provides better classification accuracy, precision, recall, and F1-score for rice leaf disease detection?

From the experiment results, the Custom CNN ended up with better results across the metrics we looked at, even though both models were really set up and trained inside

this study the same way. So, i t was pretty clear that the Custom CNN was the most effective model among the options that we tested, but we should not carry that conclusion too far, like past the exact setup we used here.

RQ4: Which disease classes are most difficult to classify correctly, and why?

The downy_mildew and brown_spot classes were the most difficult for the Custom CNN to classify correctly, with F1-scores of 0.518 and 0.599 respectively. Both classes are comparatively underrepresented in the dataset and have visual symptoms that overlap with other disease categories, both likely contributing factors.

RQ5: What are the strengths and limitations of the developed deep learning-based rice leaf disease classification framework?

The main strong point of this proposed framework is that it can automatically sort rice leaf disease images with pretty strong accuracy using the Custom CNN, along with a reproducible implementation and a somewhat clear description of an experiment that didn't work. That said there are some limits like the transfer learning setup being misconfigured, class imbalance problems, and no real explainability analysis, plus it relies on just one dataset and one particular train test split.

5.4 Comparison with Previous Studies

The findings in this study are sort of aligned with the increasing pile of evidence, showing how deep learning methods are quite effective for rice and paddy disease classification, as it was reviewed in Chapter 2. Earlier work has already stated that Convolutional Neural Networks, transfer learning designs, and also hybrid deep learning approaches can achieve strong classification results, some even mentioning accuracies above 90% and some above 98% too. For instance, the EfficientNetB4-based work reported 100% accuracy, while the hybrid deep learning framework mentioned 98.86% accuracy, which sounds pretty impressive.

In comparison, the Custom CNN accuracy of 76.18% from this study looks lower, and it can be plausibly explained by differences in dataset size, the number of classes, and also the class balance, because a number of those top-performing studies either relied on smaller, more carefully curated datasets, or they handled fewer disease categories than the ten-class Paddy Doctor dataset used here, which is moderately imbalanced, overall.

More importantly for this study's central finding, the result that a transfer learning model did worse than a model trained from scratch on rice and paddy imagery is not without precedent in the published literature. There was a separate, peer-reviewed benchmarking study, where on a paddy variety dataset, EfficientNetB0 and ResNet18 with pretrained weights ended up with lower final classification accuracy than the very same architectures trained from scratch, even though they still looked smoother, and

kind of more stable during training. The authors said this could happen because pre-trained natural image features do not quite get the fine grained visual differences you need for paddy related classification. So yeah, this gives external, independent support that transfer learning can plausibly underperform for this family of tasks, and it's separate from the specific preprocessing bug this study points to in its own EfficientNetB0 experiment. But still, the size of the underperformance seen here (16.91% accuracy) is way more intense than you'd expect from limited feature transferability alone. That's exactly why the preprocessing mismatch explanation from Section 5.2 has to be used, to properly explain the outcome.

5.5 Limitations

Even though the proposed rice leaf disease classification framework managed to show promising outcomes for the Custom CNN, some limitations should still be kept in mind while interpreting the findings. At least, the transfer learning setup was kinda not configured correctly. The EfficientNetB0 number that is reported in this work actually seems to come from a preprocessing mistake, namely using pixel values scaled in $[0, 1]$ while the backbone is expecting inputs in the $[0, 255]$ range. So its not really a clean check of the architecture's real potential on this particular task. A fixed experiment, the one laid out in Section 5.5, is needed before anything strong can be concluded about how effective transfer learning truly is for rice leaf disease classification.

Another issue is related to class imbalance. The dataset has an approximate 5:1 ratio between the biggest class and the smallest one, and that likely pushed down the performance for less represented categories like bacterial_panicle_blight and bacterial_leaf_streak. Also, the evaluation is based on only one stratified train/validation/test split, not k-fold cross validation. As a result, the reported metrics might include some extra variance that a cross validated process could better measure and expose.

Another limitation concerns explainability and model interpretability. Although the study discussed the importance of Explainable Artificial Intelligence techniques like Grad-CAM and SHAP, those methods were not actually implemented in the present experimental framework. So, as a consequence , the internal decision making processes of the developed models weren't directly checked or verified. In addition, only the Paddy Doctor dataset was used, and the performance on rice leaf images gathered from other regions, with different cameras, or under alternate lighting conditions has not been evaluated. Also, the EfficientNetB0 backbone was kept fully frozen during the whole training period, which means that even if a corrected version of the experiment was run, it still would not yet show the advantages of later fine-tuning the deeper layers.

5.6 Future Work

The findings in this study kinda show the promise of deep learning methods for automated rice leaf disease classification, but there are still some chances for more tuning and also mild correction of the suggested framework. The fastest next step, I mean the most immediate, is to re-run the EfficientNetB0 experiment using the right preprocessing setup, specifically `tf.keras.applications.efficientnet.preprocess_input`, or alternatively get rid of the $[0, 1]$ rescaling part and just give the model the raw $[0, 255]$ pixel values, so the comparison stays actually fair and more corrected versus the Custom CNN.

Another thing worth doing next is testing other transfer learning backbones besides EfficientNetB0, like MobileNetV2 and ResNet50, and keeping them under that same corrected preprocessing pipeline. This is to check whether the weaker performance we saw is only tied to EfficientNetB0 or if it shows a wider theme for this dataset, which kinda matches the mixed transfer learning outcomes often mentioned in other paddy disease studies.

Another promising area for future investigation is to incorporate Explainable Artificial Intelligence, like XAI techniques. Methods such as Grad-CAM and SHAP they can give visual explanations for model predictions, and help researchers kind of figure out which specific image regions contribute most strongly to classification decisions, for every disease class. If explainability is integrated, it would make things more transparent, it also could help with model validation, and in general increase confidence in deployment scenarios where decision reliability is really critical

Future work might also try to deal with the class imbalance that was identified in Section 3.2 through class-weighting or perhaps oversampling methods, especially for minority classes like bacterial_panicle_blight and bacterial_leaf_streak. In addition, fine-tuning the later layers of the pretrained backbone instead of freezing it completely, could be useful after the preprocessing issue has been corrected. Additional research that evaluates on rice leaf images gathered from sources outside the Paddy Doctor dataset, and deployment of the trained Custom CNN behind a simple web or mobile interface, would make the overall framework more robust, and more practical, for real-world use.

6 Conclusion

This research sort of examined how deep learning can be used for automated rice leaf disease classification, based on the publicly available Paddy Doctor dataset, which includes 10,407 images across ten classes, nine diseases plus one healthy group. The main goal was to check how well a Custom Convolutional Neural Network and an EfficientNetB0 transfer learning setup perform when it comes to correctly assigning rice leaf disease images into their right categories. An overall experimental workflow was put together, including

things like dataset preprocessing and image augmentation, then model training, followed by performance assessment and then a sort of side by side comparison. In the end, the outcomes showed that the Custom CNN managed to pick up useful disease-specific visual patterns, reaching test accuracy of 76.18% and a macro F1-score of 0.714 over all ten classes. Meanwhile the EfficientNetB0 transfer learning model lagged far behind, reaching only 16.91% accuracy.

Critically , this study did not just stop at reporting the EfficientNetB0 output as some kind of plain negative result. On closer inspection, especially of the classification report and confusion matrix , it turned out the model was basically collapsing into predicting the majority class for almost every test image . Then, further investigation traced the odd behavior back to a specific preprocessing mismatch, which was also actually correctable: the pipeline uses a $[0, 1]$-rescaled input flow, but the pretrained EfficientNetB0 backbone expects the values in the $[0, 255]$ range. So the real point of this report is important. It does not simply say pretrained transfer learning is "not suited" for rice disease classification, instead it points to a precise cause for the underperformance, and it describes the corrected experiment you would run to properly judge the architecture.

The work also sorta emphasized how much preprocessing choices , class-level evaluation metrics, and systematic evaluation approaches matter when you want dependable deep learning in agricultural use cases. Using comparative analysis grounded both in this study's own results and in outside literature about transfer learning for paddy - related classification problems, the research argues that the model plus the full pipeline setup can influence outcomes just as strongly as the architecture itself, especially for performance in real-world settings

Overall the research did manage to hit its objectives and also answered the proposed research questions . The findings do confirm that deep learning-based image classification systems can offer a dependable foundation for automated rice disease monitoring and early diagnosis support. Even so , some limitations remain , like the misconfigured transfer learning experiment , the dataset scope, and explainability considerations. Still, the results provide solid evidence for continued exploration of artificial intelligence technologies for agricultural monitoring and food security. Future research that includes a corrected transfer learning pipeline, more pretrained architectures, class-imbalance handling, and explainable AI techniques can further improve the effectiveness and day-to-day practical applicability of automated rice leaf disease detection systems.

References

[1] Pandarasamy Arjunan and Petchiammal. (2022). "Paddy Doctor: Paddy Disease Classification." Kaggle. Available at: `https://www.kaggle.com/competitions/paddy-disease-classification`

[2] Petchiammal et al. "Paddy Doctor: A Visual Image Dataset for Automated Paddy Disease Classification and Benchmarking." IEEE DataPort. Available at: https://paddydoc.github.io

[3] Tan, M. and Le, Q. (2019). "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks." *Proceedings of the 36th International Conference on Machine Learning (ICML)*.

[4] He, K., Zhang, X., Ren, S., and Sun, J. (2016). "Deep Residual Learning for Image Recognition." *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778.

[5] LeCun, Y., Bengio, Y., and Hinton, G. (2015). "Deep learning." *Nature*, 521(7553), pp. 436–444.

[6] Pan, S. J. and Yang, Q. (2010). "A Survey on Transfer Learning." *IEEE Transactions on Knowledge and Data Engineering*, 22(10), pp. 1345–1359.

[7] Mohanty, S. P., Hughes, D. P., and Salathé, M. (2016). "Using Deep Learning for Image-Based Plant Disease Detection." *Frontiers in Plant Science*, 7.

[8] "A systematic review of deep learning applications for rice disease diagnosis: current trends and future directions." *Frontiers in Computer Science* (2024). Available at: https://www.frontiersin.org/journals/computer-science/articles/10.3389/fcomp.2024.1452961/full

[9] Padhi, J. et al. "Paddy Leaf Disease Classification Using EfficientNet B4 with Compound Scaling and Swish Activation: A Deep Learning Approach."

[10] "An automated hybrid deep learning framework for paddy leaf disease identification and classification." *Scientific Reports* (2025). Available at: https://www.nature.com/articles/s41598-025-08071-6

[11] "Deep learning-based automatic diagnosis of rice leaf diseases using ensemble CNN models." PMC. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC12307908/

[12] "Experimental Comparison of Light-Weight and Deep CNN Models Across Diverse Datasets." arXiv:2601.03463.

[13] "Paddy Leaf Symptom-based Disease Classification Using Deep CNN with ResNet-50." *International Journal of Advanced Science Computing and Engineering* (2022).

[14] Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2017). "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization." *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.

[15] Lundberg, S. M. and Lee, S. I. (2017). "A Unified Approach to Interpreting Model Predictions." *Advances in Neural Information Processing Systems*, Vol. 30.

[16] TensorFlow Developers. "TensorFlow Documentation." Available at: `https://www.tensorflow.org`

[17] Keras Developers. "Keras Documentation." Available at: `https://keras.io`